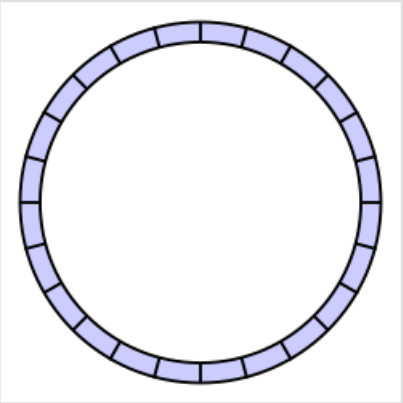
WIKIPEDIA
25 years of the free encyclopedia

Circular buffer

In computer science, a **circular buffer**, **circular queue**, **cyclic buffer** or **ring buffer** is a data structure that uses a single, fixed-size buffer as if it were connected end-to-end. This structure lends itself easily to buffering data streams.[1] There were early circular buffer implementations in hardware.[2][3]



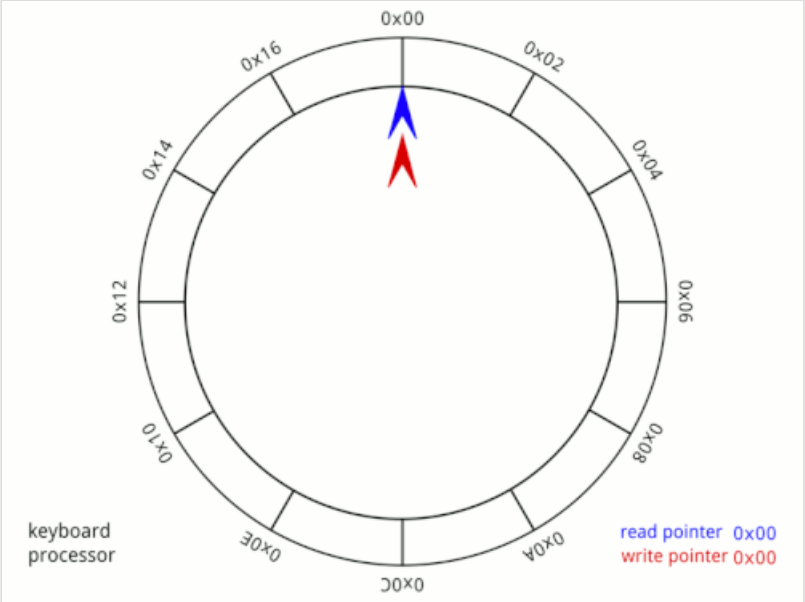
A ring showing, conceptually, a circular buffer. This visually shows that the buffer has no real end and it can loop around the buffer. However, since memory is never physically created as a ring, a linear representation is generally used as is done below.

Overview

A circular buffer first starts out empty and has a set length. In the diagram below is a 7-element buffer:



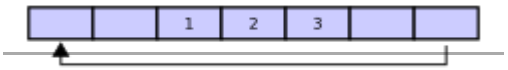
Assume that 1 is written in the center of a circular buffer (the exact starting location is not important in a circular buffer):



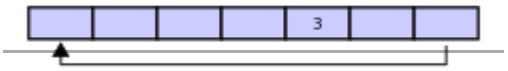
A 24-byte keyboard circular buffer. When the write pointer is about to reach the read pointer—because the microprocessor is not responding—the buffer stops recording keystrokes. On some computers a beep would be played.



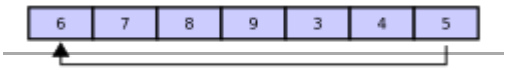
Then assume that two more elements are added to the circular buffer — 2 & 3 — which get put after 1:



If two elements are removed, the two oldest values inside of the circular buffer would be removed. Circular buffers use FIFO (*first in, first out*) logic. In the example, 1 & 2 were the first to enter the circular buffer, they are the first to be removed, leaving 3 inside of the buffer. When elements get removed, only the read-pointer moves to the next element. Removed elements are still in the buffer, waiting to get overwritten by new elements.



If the buffer has 7 elements, then it is completely full:



A property of the circular buffer is that when it is full and a subsequent write is performed, then it starts overwriting the oldest data. In the current example, two more elements — A & B — are added and they *overwrite* the 3 & 4:



Alternatively, the routines that manage the buffer could prevent overwriting the data and return an error or raise an exception. Whether or not data is overwritten is up to the semantics of the buffer routines or the application using the circular buffer.

Finally, if two elements are now removed then what would be removed is **not** A & B, but 5 & 6 because 5 & 6 are now the oldest elements, yielding the buffer with:



Uses

The useful property of a circular buffer is that it does not need to have its elements shuffled around when one is consumed. (If a non-circular buffer were used then it would be necessary to shift all elements when one is consumed.) In other words, the circular buffer is well-suited as a FIFO (*first in, first out*) buffer while a standard, non-circular buffer is well suited as a LIFO (*last in, first out*) buffer.

Circular buffering makes a good implementation strategy for a queue that has fixed maximum size. Should a maximum size be adopted for a queue, then a circular buffer is a completely ideal implementation; all queue operations are constant time. However, expanding a circular buffer requires

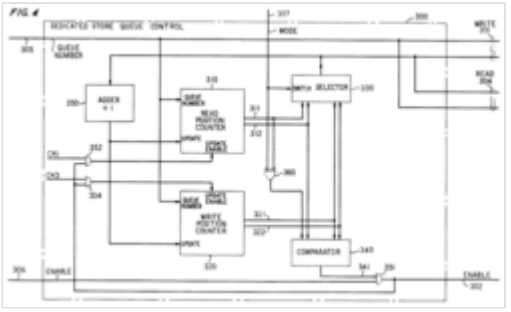
shifting memory, which is comparatively costly. For arbitrarily expanding queues, a linked list approach may be preferred instead.

In some situations, an overwriting circular buffer can be used, e.g. in multimedia. If the buffer is used as the bounded buffer in the producer–consumer problem then it is probably desired for the producer (e.g., an audio generator) to overwrite old data if the consumer (e.g., the sound card) is unable to momentarily keep up. Also, the LZ77 family of lossless data compression algorithms operates on the assumption that strings seen more recently in a data stream are more likely to occur soon in the stream. Implementations store the most recent data in a circular buffer.

Circular buffer mechanics

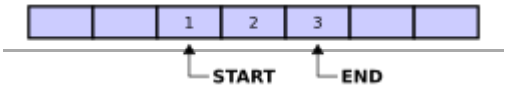
A circular buffer can be implemented using a pointer and four integers:[4]

- buffer start in memory
- buffer capacity (length)
- "write to" buffer index (end)
- "read from" buffer index (start)

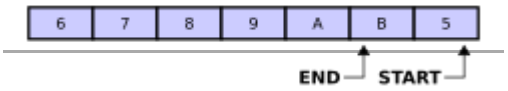


Circular buffer implementation in hardware, US patent 3979733, fig4

This image shows a partially full buffer with Length = 7:



This image shows a full buffer with four elements (numbers 1 through 4) having been overwritten:



In the beginning the indexes end and start are set to 0. The circular buffer write operation writes an element to the end index position and the end index is incremented to the next buffer position. The circular buffer read operation reads an element from the start index position and the start index is incremented to the next buffer position.

The start and end indexes alone are not enough to distinguish between buffer full or empty state while also utilizing all buffer slots,[5] but can be if the buffer only has a maximum in-use size of Length − 1.[6] In this case, the buffer is empty if the start and end indexes are equal and full when the in-use size is Length − 1. Another solution is to have another integer count that is incremented at a write operation and decremented at a read operation. Then checking for emptiness means testing count equals 0 and checking for fullness means testing count equals Length.[7]

The following source code is a minimal C++ implementation. `push()` puts an item in the buffer, `pop()` gets an item from the buffer. Both functions take care about the capacity of the buffer:

```
import std;
```

```cpp
using std::array;
using std::runtime_error;

template <typename T, size_t N>
class CircularBuffer {
private:
    array<T, N> data{};
    size_t head = 0;
    size_t tail = 0;
    size_t size = 0;
public:
    bool isEmpty() const noexcept {
        return size == 0;
    }

    bool isFull() const noexcept {
        return size == N;
    }

    size_t getSize() const noexcept {
        return size;
    }

    void push(const T& value) {
        if (isFull()) {
            throw runtime_error("Buffer is full");
        }
        data[tail] = value;
        tail = (tail + 1) % N;
        ++size;
    }

    T pop() {
        if (isEmpty()) {
            throw runtime_error("Buffer is empty");
        }
        T value = data[head];
        head = (head + 1) % N;
        --size;
        return value;
    }

    T peek() const {
        if (isEmpty()) {
            throw runtime_error("Buffer is empty");
        }
        return data[head];
    }
};

int main() {
    CircularBuffer<int, 3> buffer;
    buffer.push(1);
    buffer.push(2);
    buffer.push(3);

    std::println("{}", buffer.pop()); // prints 10
    buffer.push(4);

    while (!buffer.isEmpty()) {
        std::println("{}", buffer.pop());
    }
}
```

Optimization

A circular-buffer implementation may be optimized by mapping the underlying buffer to two contiguous regions of virtual memory.[8] (Naturally, the underlying buffer‘s length must then equal some multiple of the system’s page size.) Reading from and writing to the circular buffer may then be carried out with

greater efficiency by means of direct memory access; those accesses which fall beyond the end of the first virtual-memory region will automatically wrap around to the beginning of the underlying buffer. When the read offset is advanced into the second virtual-memory region, both offsets—read and write—are decremented by the length of the underlying buffer.

Fixed-length-element and contiguous-block circular buffer

Perhaps the most common version of the circular buffer uses 8-bit bytes as elements.

Some implementations of the circular buffer use fixed-length elements that are bigger than 8-bit bytes—16-bit integers for audio buffers, 53-byte ATM cells for telecom buffers, etc. Each item is contiguous and has the correct data alignment, so software reading and writing these values can be faster than software that handles non-contiguous and non-aligned values.

Ping-pong buffering can be considered a very specialized circular buffer with exactly two large fixed-length elements.

The *bip buffer* (bipartite buffer) is very similar to a circular buffer, except it always returns contiguous blocks which can be variable length. This offers nearly all the efficiency advantages of a circular buffer while maintaining the ability for the buffer to be used in APIs that only accept contiguous blocks.[9]

Fixed-sized compressed circular buffers use an alternative indexing strategy based on elementary number theory to maintain a fixed-sized compressed representation of the entire data sequence.[10]

References

1. Arpaci-Dusseau, Remzi H.; Arpaci-Dusseau, Andrea C. (2014), *Operating Systems: Three Easy Pieces [Chapter: Condition Variables, figure 30.13]* (http://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf) (PDF), Arpaci-Dusseau Books
2. Hartl, Johann (17 October 2011). "Impulswiederholer - Telephone Exchange (video)" (https://www.youtube.com/watch?v=_xI9tXi-UNs). Youtube. Retrieved 15 December 2021.
3. Fraser, Alexander Gibson. "US patent 3979733 Digital data communications system packet switch" (https://patents.google.com/patent/US3979733A/en). US States Patent. Retrieved 15 December 2021.
4. Liu, Z.; Wu, F.; Das, S.K. (2021). *Wireless Algorithms, Systems, and Applications: 16th International Conference, WASA 2021, Nanjing, China, June 25–27, 2021, Proceedings, Part II* (https://books.google.com/books?id=si1CEAAAQBAJ&pg=PA117). Lecture Notes in Computer Science. Springer International Publishing. p. 117. ISBN 978-3-030-86130-8. Retrieved 2023-09-04.
5. Chandrasekaran, Siddharth (2014-05-16). "Implementing Circular/Ring Buffer in Embedded C" (https://embedjournal.com/implementing-circular-buffer-embedded-c/). *Embed Journal*. EmbedJournal Team. Archived (https://web.archive.org/web/20170211031659/http://embedjournal.com/implementing-circular-buffer-embedded-c/) from the original on 11 February 2017. Retrieved 14 August 2017.
6. Circular buffers (https://www.kernel.org/doc/Documentation/circular-buffers.txt#:~:text=A%20circular%20buffer%20is%20a,next%20item%20in%20the%20buffer) kernel.org
7. Morin, Pat. "ArrayQueue: An Array-Based Queue" (http://opendatastructures.org/ods-python/2_3_ArrayQueue_Array_Based_.html). *Open Data Structures (in pseudocode)*. Archived

(https://web.archive.org/web/20150831023453/http://opendatastructures.org/ods-python/2_3_ArrayQueue_Array_Based_.html) from the original on 31 August 2015. Retrieved 7 November 2015.

8. Mike Ash (2012-02-17). "mikeash.com: Friday Q&A 2012-02-17: Ring Buffers and Mirrored Memory: Part II" (https://www.mikeash.com/pyblog/friday-qa-2012-02-17-ring-buffers-and-mirrored-memory-part-ii.html). *mikeash.com*. Archived (https://web.archive.org/web/20190111054903/https://www.mikeash.com/pyblog/friday-qa-2012-02-17-ring-buffers-and-mirrored-memory-part-ii.html) from the original on 2019-01-11. Retrieved 2019-01-10.
9. Simon Cooke (2003), "The Bip Buffer - The Circular Buffer with a Twist" (https://web.archive.org/web/20260204171614/http://www.codeproject.com/articles/The-Bip-Buffer-The-Circular-Buffer-with-a-Twist)
10. Gunther, John C. (March 2014). "Algorithm 938: Compressing circular buffers". *ACM Transactions on Mathematical Software*. **40** (2): 1–12. doi:10.1145/2559995 (https://doi.org/10.1145%2F2559995). S2CID 14682572 (https://api.semanticscholar.org/CorpusID:14682572).

External links

- CircularBuffer at the Portland Pattern Repository
- Boost:
  - Templated Circular Buffer Container (https://www.boost.org/doc/libs/release/doc/html/circular_buffer.html): circular_buffer/base.hpp (https://github.com/boostorg/circular_buffer/blob/develop/include/boost/circular_buffer/base.hpp)
  - Synchronized Bounded Queue (https://www.boost.org/doc/libs/release/doc/html/thread/sds.html#thread.sds.synchronized_queues.ref.sync_bounded_queue_ref): sync_bounded_queue.hpp (https://github.com/boostorg/thread/blob/develop/include/boost/thread/concurrent_queues/sync_bounded_queue.hpp)
- CB in Linux kernel (https://www.kernel.org/doc/html/latest/core-api/circular-buffers.html)
- CB in DSP (http://www.dspguide.com/ch28/2.htm)
- Circular queue in C (http://www.martinbroadhurst.com/cirque-in-c.html) Archived (https://web.archive.org/web/20181029235921/http://www.martinbroadhurst.com/cirque-in-c.html) 2018-10-29 at the Wayback Machine

Retrieved from "https://en.wikipedia.org/w/index.php?title=Circular_buffer&oldid=1365711505"